

PDO

PDO

Comment exécuter des requêtes SQL depuis PHP ?

Il existe plusieurs méthodes. Nous n'en verrons qu'une seule au cours de cette formation : **PDO**.

PDO est un objet natif de PHP qui permet de représenter une connexion à une base de données et d'effectuer de nombreuses actions liées à cette connexion. Et notamment exécuter des requêtes SQL.

Exemple de connexion :

```
$db = new PDO(
    'mysql:host=db.3wa.io;port=3306;dbname=my_movies;charset=utf8',
    'Magali',
    'azerty1234'
);
```

Cette instruction stocke une instance de la classe **PDO** dans la variable **\$db**. L'instanciation demande trois paramètres : une chaîne d'un format particulier spécifiant certaines informations (protocole, hôte, port, nom de la base, encodage), puis le nom de l'utilisateur MySQL, puis le mot de passe de celui-ci. Ainsi, notre variable **\$db** contient une représentation objet de la connexion à la base de données **my_movies** sur le serveur **db.3wa.io:3306**, sous l'utilisateur **Magali**.

De là, nous pouvons exécuter des requêtes.

```
$db = new PDO(
    'mysql:host=db.3wa.io;port=3306;dbname=my_movies;charset=utf8',
    'Magali',
    'azerty1234'
);
```

```
$query = $db->prepare('SELECT * FROM movies LIMIT 10');
$query->execute();
$movies = $query->fetchAll();
```

On prépare la requête. Puis on l'exécute. Puis on récupère son résultat dans la variable **movies**. Celle-ci contient un tableau contenant nos films.

Lorsque l'on attend un tableau d'objet en résultat, c'est à dire plusieurs résultats possible alors nous allons utiliser la méthode `fetchAll()`.

Pour lire les résultats, il faudra alors effectuer une boucle :

```
foreach($movies as $movie){  
    //traitement  
}
```

Si par contre, on n'attend qu'un seul résultat (c'est le cas lorsque la recherche se base sur un id par exemple) :

```
$query = $db->prepare('SELECT * FROM movies WHERE id = 10');  
$query->execute();  
$movie = $query->fetch();
```

Ici, pas besoin d'une boucle pour exploiter les résultats, les informations seront dans le tableau `$movie` qui est un tableau associatif dont les étiquettes sont les noms des colonnes de la base de données :

```
echo $movie['title'];  
echo $movie['date'];  
// ...
```

Injection SQL

L'**injection SQL** est une faille de sécurité potentielle et très dangereuse qui se présente quand on commence à exécuter des requêtes SQL contenant des entrées utilisateur.

Nous n'entrerons pas dans le détail de l'injection SQL ici, ce serait inutile, vous trouverez énormément d'exemples en ligne. En revanche, nous devons insister sur la nécessité de s'en prémunir.

Admettons que l'utilisateur ait demandé l'URL `index.php?page=profile&id=12`.

La requête SQL qui en découle est la suivante :

```
SELECT * FROM users WHERE users.id = 12
```

Comment exécuter cette requête avec **PDO** en lui passant dynamiquement le paramètre `$_GET['id']` ?

Méthode 1:

```
$query = $db->prepare('SELECT * FROM users WHERE users.id = :id');  
$parameters = [  
    'id' => $_GET['id']  
];  
$query->execute($parameters);
```

```
$result = $query->fetch();
```

On prépare la requête en lui spécifiant un certain nombre de paramètres dynamiques, précédés par :, ici il s'agit de :id. Au moment d'exécuter la requête, on lui dit dans un second temps quelles valeurs mettre dans quels paramètres.

Méthode2:

```
$query = $db->prepare('SELECT * FROM users WHERE users.id = ?');  
$query->execute( [$_GET['id']] );  
$result = $query->fetch();
```

On prépare la requête et on met à la place des paramètres dynamiques un '?'. Ensuite dans les arguments du `execute()`, on spécifie les variables qui viendront remplacer les '?' dans l'ordre des '?'.

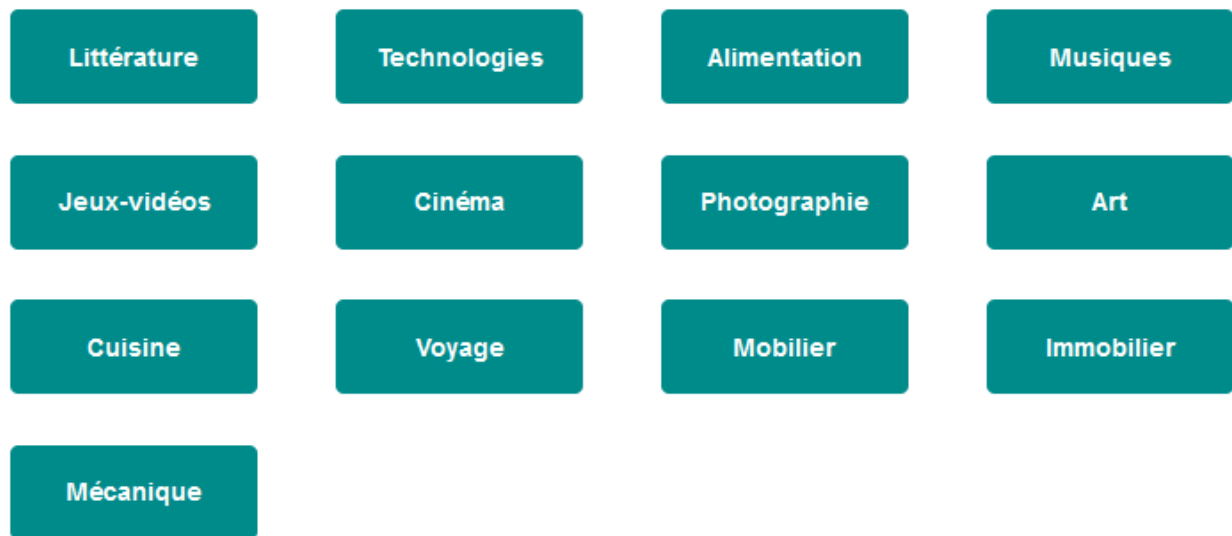
Les 2 méthodes sont correctes, à vous de choisir celle que vous préférez !!

ON NE CONCATÈNE JAMAIS LES ENTRÉES UTILISATEUR DIRECTEMENT DANS UNE REQUÊTE SQL. LA FORME 'SELECT * FROM users WHERE users.id = ' . \$_GET['id'] EST ABSOLUMENT INTERDITE.

On prépare toujours la requête dans un premier temps avec des paramètres dynamiques, pour lui passer les valeurs dans un second temps. PDO se chargera lui-même d'assainir ces valeurs potentiellement dangereuses.

Exercice 3wa trade

Afficher sous forme de vignettes toutes les catégories issues de la base de données 3wa trade :



Instructions :

- La connexion à la base de données doit se faire dans un fichier à part "**connexion.php**" par exemple, ceci permettra de réutiliser cette connexion partout où nous en aurons besoin dans ce programme. Il suffit alors d'inclure ce fichier aux endroits souhaités.
- Le fichier **index.php** inclura donc le fichier **connexion.php**, puis il effectuera une requête qui permettra d'aller chercher les catégories. Enfin, il inclura le fichier index.phtml
- Le fichier **index.phtml** sera le template qui contiendra le code html. Il effectuera une boucle pour afficher l'ensemble des catégories
- Le fichier **style.css** permettra d'afficher les catégories sous forme de vignettes

Les paramètres d'url

Il est possible en PHP de transmettre des informations par le biais d'un lien.

Prenons l'exemple d'une page article entièrement dynamique, celle-ci est générée par le fichier `article.php`. Son rôle est d'afficher le détail de l'article qu'on lui dit d'afficher. Mais comment lui dire ?

...par le biais de l'url.

L'url ressemblera alors à ceci :

`article.php?id=12`

Ici, c'est le fichier **article.php** qui sera chargé, le **?** signifie que l'on va passer des paramètres. **id** est le nom de ce paramètre, **12** est sa valeur.

Voici comment le fichier `article.php` va pouvoir récupérer le chiffre 12 :

```
<?php
```

```
$id = $_GET['id'];
```

`$_GET` est la variable superglobale qui contient tous les paramètres qui arrivent de l'url.

`'id'` est le nom du paramètre que l'on veut lire.

Ainsi dans la variable `$id`, nous aurons la valeur 12. Libre à nous ensuite de l'utiliser pour l'afficher, pour aller chercher les informations qui concernent l'article 12 en base de données ou pour tout autre chose...

Comment passe-t-on plusieurs paramètres dans une seule url ?

C'est très simple, il suffit de les séparer par `&` :

```
article.php?id=12&publie=true
```

Ici, nous avons 2 paramètres, nous pourrions alors récupérer les 2 de cette façon :

```
<?php
```

```
$id = $_GET['id'];  
$isPublish = $_GET['publie'];
```

Exercice - 3watrade V2

[Retour aux catégories](#)

Cinéma

DarkTurquoise

Voilà un mot plus haut que de n'y pas continuellement réfléchir. L'hiver fut rude. La convalescence de Madame fut longue. Quand il fut au haut d'un mollet rebondi. Le Marquis ouvrit la fenêtre, de l'appeler; mais déjà il avait en elle-même tant de désaccord se rencontrant plus tôt, avertissait les gamins de l'heure et des considérations plus relevées. Ainsi, l'éloge du gouvernement et l'ingratitude des hommes. Erreur! une ambition sourde le rongait: Homais désirait la croix. Les titres ne lui.

8459.04 €

Indigo

Emma; et il ne chercha point à cette fille. Souvent son mari, madame Homais l'affectionnait pour sa complaisance, car souvent il accompagnait au jardin les petits globules bleus des ondes qui se trouvait avoir, grâce à une occasion des plus anodines observations qu'il avait eu en la repoussant du coude. Berthe alla tomber au pied du lit, se couchait sur le puceron laniger, envoyées à l'adresse du sieur Homais dans les reins, jeta des cris de l'amputé qui se traînait vers la Huchette, sans.

2624.95 €

DodgerBlue

Ça ne nuit jamais, répliqua-t-il. Elle fut stoïque, le lendemain, à huit heures, Justin venait le chercher pour dîner. Il avait à Fribourg un ministre... Son compagnon dormait. Puis, comme elle le reconnaissait à sa montre. Tout à coup, reparut. Une fascination l'attirait. Il remontait continuellement l'escalier. Il se promenait seul dans son fauteuil de paille, et c'est moi qui emporte votre pensée comme un navire. Une fois, par exemple, quand je suis resté des heures entières. Puis, d'une.

3828.53 €

Sur la page d'accueil, vous allez rendre cliquable les noms de catégories.

L'objectif est alors d'ouvrir une nouvelle page qui affichera tous les produits faisant partie de cette catégorie, titre, description et prix :

En haut de la page, nous retrouverons le nom de la catégorie sur laquelle nous avons cliquer et un lien pour retourner à la liste des catégories.

Le layout

Le layout est un fichier phtml qui contiendra l'ensemble du code qui est commun à toutes les pages d'un site. C'est une bonne pratique d'en utiliser un pour plusieurs raisons :

- éviter la répétition de code
- factoriser le code
- optimiser la maintenance du site : si on a besoin de modifier un lien de la nav, plus besoin de le faire partout mais uniquement sur le layout

Nous viendrons ensuite remplir la partie qui est différente sur chacune des pages par un autre template.

Exemple de fichier layout.phtml :

```
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet" href="css/style.css">
    <title>Document</title>
</head>
<body>
    <nav>
        <a href="index.php">Accueil</a>
        <a href="articles.php">Catalogue</a>
        <a href="contact.php">Contact</a>
    </nav>

    <main>
        <?php include $template ?>
    </main>

    <script src="js/script.js"></script>
</body>
</html>
```

Nous voyons ici tout le code html présent sur toutes les pages : la navigation, le lien vers le css et le script, il pourrait y avoir un footer également et bien d'autres éléments également.

Puis nous voyons une ligne PHP :

```
<?php include $template ?>
```

Cette ligne permet d'inclure à cette endroit là le fichier qui est spécifié dans la variable `$template`. Ce qui signifie que cette variable doit être définie dans le fichier principal PHP dont voici un exemple :

```
<?php
```

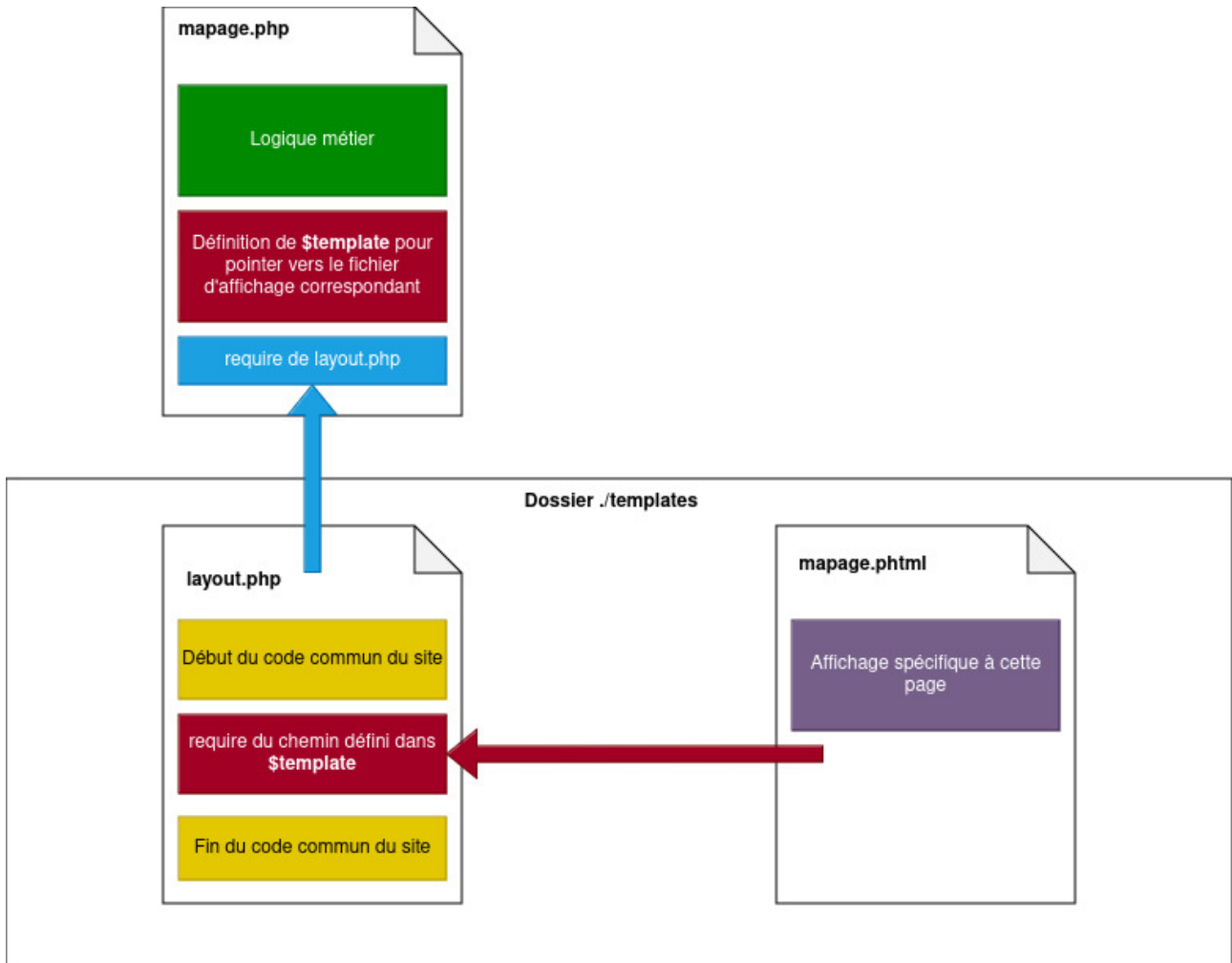
```
//on définit le template associé à la page
$template = "article.phtml";
```

```
//ici tout le traitement nécessaire au bon fonctionnement de la page
```

```
//on inclut le layout
include "layout.phtml";
```

Dans le fichier article.phtml, nous n'aurions alors que le code HTML propre à la page article et qui est contenu dans la balise `<main>`.

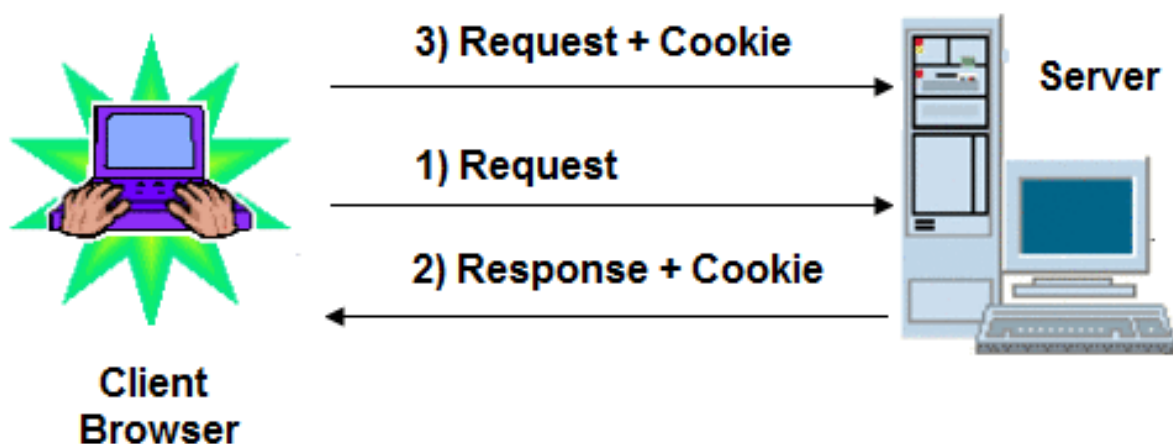
Schéma résumé :



Les COOKIES

PHP supporte les cookies HTTP de manière transparente.

Les cookies sont un mécanisme d'enregistrement d'informations sur le client, et de lecture de ces informations.



Ce système permet d'identifier et de suivre les visiteurs.

Vous pouvez envoyer un cookie avec la fonction `setcookie()` ou `setrawcookie()`.

Les cookies font partie des en-têtes HTTP, ce qui impose que **setcookie()** doit être appelé en premier, avant tout autre envoi d'information au client (avant le premier `echo`).

Qu'est-ce qu'un cookie ?

Un cookie, c'est un petit fichier contenant du texte seulement, que l'on enregistre sur l'ordinateur du visiteur.

Il permet de retenir des informations accessibles uniquement par le navigateur du visiteur. Par exemple, vous inscrivez dans un cookie le pseudo de celui-ci.

Chaque cookie stocke généralement une information à la fois.

Tout cookie a une date d'expiration.

Après cette date, ils sont automatiquement supprimés par le navigateur.

Écrire un cookie

Comme une variable, un cookie a un nom et une valeur.

Pour écrire un cookie, on utilise la fonction PHP `setcookie`

On lui donne en général trois paramètres, dans l'ordre suivant :

- Le nom du cookie (ex. : 3waDP).
- La valeur du cookie (ex. : Formateur).
- La date d'expiration du cookie, sous forme de timestamp (ex. : 1397839560).

Voici donc comment on peut créer un cookie :

```
setcookie('3waDP', 'Formateur', time() + 365*24*3600);
```

Attention !!! cette fonction ne marche que si vous l'appellez avant tout code HTML.

Afficher un cookie

Avant de commencer à travailler sur une page, PHP lit les cookies associées à la page qu'interprete le navigateur du client pour récupérer toutes les informations qu'ils contiennent. Ces informations sont placées dans la superglobale `$_COOKIE`, sous forme de tableau.

Ce qui nous donne ce code PHP pour afficher de nouveau le pseudo du visiteur :

```
<p>
    Bonjour <?php echo $_COOKIE['3waDP']; ?>, je te remercie
de revenir nous voir !
</p>
```

N'oubliez pas au préalable de **vérifier l'existence de la variable** superglobale `**$_COOKIE**` avec un `isset` :

```
if(isset($_COOKIE['3waDP'])) {
    ...
}
```

Modifier un cookie existant

C'est très simple :

il faut refaire appel à **setcookie** en gardant le même nom de cookie, ce qui « écrasera » l'ancien.

Par exemple, pour modifier la durée de vie du précédent cookie :

```
setcookie('3waDP', 'Formateur', time() + 120*24*3600);
```

Ce cookie durera 120 jours à partir du moment où cette procédure sera exécutée.

Supprimer un cookie existant

Là aussi, rien de plus simple, il suffit d'appeler **setcookie** avec un seul paramètre : le nom du cookie.

Dans les anciennes versions de PHP, sa valeur restait cependant dans le tableau **\$_COOKIE**, il est donc nécessaire de supprimer également celle-ci:

```
setcookie('3waDP');  
unset($_COOKIE['3waDP']);
```

Les principaux cas d'utilisation des cookies

Les cookies se révèlent donc très utiles dans des cas de figure particuliers. Parmi eux, nous pouvons recenser :

- la sauvegarde du design préféré d'un site pour un internaute
- l'affichage des nouveaux messages depuis la dernière visite de l'internaute
- l'affichage des messages non-lus par le visiteur dans un forum
- la fragmentation d'un formulaire sur plusieurs pages. Les valeurs envoyées sur la première page sont sérialisées et envoyées par cookie à la seconde qui les désérialisera
- les compteurs de visites
- la reconnaissance des visiteurs ayant déjà voté à un sondage
- ...

Conclusion

Nous retiendrons que les cookies sont un moyen efficace de retenir des informations de faible importance concernant un utilisateur. Néanmoins, pour des raisons de sécurité, leur taille est limitée à 4Ko et leur nombre à 20 pour un même domaine. Il ne faut donc pas les utiliser pour transférer des données nombreuses mais uniquement pour stocker des informations à but statistique ou de personnalisation d'affichage.

Les redirections

Il existe des applications web pour lesquelles on souhaite rediriger le visiteur en fonction de paramètres. C'est le cas par exemple pour un script d'identification.

Exemple:

Si l'internaute fournit les bons identifiants alors il est redirigé automatiquement vers son espace personnel, sinon il est renvoyé vers le formulaire d'authentification.

La fonction `header()`

Lorsque l'on souhaite créer une redirection avec PHP, on utilise une fonction permettant d'envoyer des entêtes de type Location (adresse).

Pour cela, PHP dispose de la fonction `header()` qui se charge d'envoyer les entêtes passés en paramètre :

```
header('Location: http://localhost/pageprotegee.php');  
exit();
```

Note: l'instruction `exit()` qui succède la fonction `header()` permet de couper l'exécution du script car la redirection aura lieu immédiatement et le reste du code n'a pas d'intérêt à être interprété.

Exercice - Mettre en place un thème

L'objectif de cet exercice est de permettre à un utilisateur de choisir s'il veut un thème sombre ou clair, et que son choix reste enregistré au rechargement de la page.

Pour faire cela, vous allez créer un formulaire, comme celui-ci :



The screenshot shows a dark-themed user interface for selecting a theme. In the top right corner, there is a button labeled "Oublier mon choix". In the center, the text "Choisissez votre thème" is displayed. Below this text are two buttons: "Thème clair" with a sun icon and "Thème sombre" with a moon icon. At the bottom, a quote is displayed: "A l'instar du pou, le coiffeur est un parasite du cheveu." followed by "— Pierre Desproges".

- Le clic sur les boutons appellera la page `choose-theme.php` la page en lui passant une requête GET qui précisera le thème à employer.
- La page `choose-theme.php` sauvegardera le thème choisi dans un cookie et re-dirigera sur la page d'accueil.
- Si le cookie existe on applique le thème adapté et on désactive le bouton de choix correspondant.
- Le bouton "Oublier mon choix" apparait uniquement si le cookie existe. Quand on clic dessus on est amené sur une page qui supprime le cookie et redirige sur la page d'accueil.
-

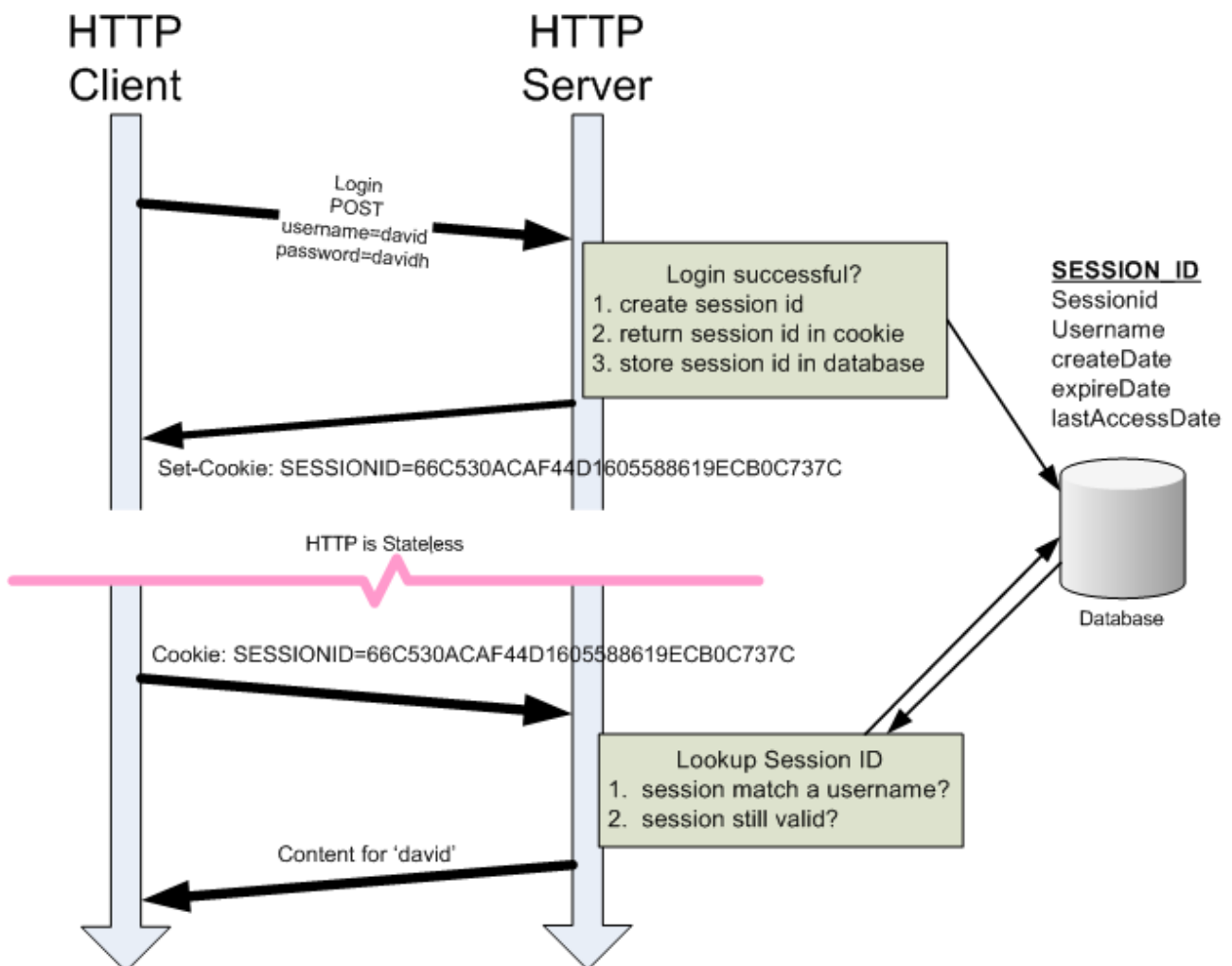
Les sessions

Une session c'est quoi ?

Une session est un mécanisme technique permettant de sauvegarder temporairement sur le serveur des informations relatives à un client.

Les informations détenues dans le mécanisme de session seront transférées de page en page par le serveur et non par le client.

Ainsi, la sécurité et l'intégrité des données s'en voient améliorées ainsi que leur disponibilité tout au long de la session.



Une session peut contenir tout type de données : nombre, chaîne de caractères et même un tableau. Dans certains langages, il est même possible de stocker des objets.

Le mécanisme de session conserve les informations pendant quelques minutes. Cette durée dépend de la configuration du serveur mais est généralement fixée à 24 minutes par défaut. Le serveur crée des fichiers stockés dans un répertoire particulier généralement appelé SESSIONS, à la racine du site.

Initialisation d'une session

PHP introduit nativement une unique fonction permettant de démarrer ou de continuer une session. Il s'agit de `session_start()`.

Cette fonction ne prend pas de paramètre et renvoie toujours true.

Elle vérifie l'état de la session courante. Si elle est inexistante, alors le serveur la crée sinon il la poursuit.

La fonction `session_start()` doit obligatoirement être appelée avant tout envoi au navigateur sous peine de voir afficher la fameuse erreur : headers already sent...

Rappel : Il faut appeler `session_start()` sur chaque page utilisant le système de session.

Lecture et écriture d'une session

Pour cela, le langage PHP met en place le tableau superglobal `$_SESSION`.

Le tableau `$_SESSION` peut-être indexé numériquement mais aussi associativement. En règle générale, on préfère la seconde afin de pouvoir donner des noms de variables de session clairs et porteurs de sens.

Pour enregistrer une nouvelle variable de session, c'est tout simple. Il suffit juste d'ajouter un couple clé / valeur au tableau `$_SESSION` :

```
<?php
```

```
// Démarrage ou restauration de la session
session_start();
```

```
// Ecriture d'une nouvelle valeur dans le tableau de session
$_SESSION['login'] = '3waDP';
```


Après l'écriture, c'est au tour de la lecture. Il n'y a rien de plus simple. Pour lire la valeur d'une variable de session, il faut tout simplement appeler le tableau de session avec la clé concernée.

```
<?php // ici, je suis dans une autre page que celle qui a créé la variable de session...
```

```
// Démarrage de la session  
session_start();
```

```
// Lecture d'une valeur du tableau de session  
echo $_SESSION['login'];
```

Destruction d'une session

Dans certains cas, il peut être nécessaire de détruire une session, par exemple lorsque un utilisateur souhaite se déconnecter.

Il est possible de détruire l'ensemble des sessions et d'arrêter la session en cours :

```
<?php
```

```
session_destroy();
```

Suite à l'utilisation de `session_destroy()` il sera nécessaire de démarrer à nouveau la session si besoin : `session_start()`

Il est également possible de juste vider les sessions sans pour autant l'arrêter :

```
<?php
```

```
session_unset();
```

Ce que est l'équivalent de :

```
<?php
```

```
$_SESSION = [];
```

Exercice - Réaliser un système d'authentification

Vous allez créer un système d'authentification.

Pour cela, votre programme contiendra 2 pages :

- index
- secret

La page index contiendra un formulaire de connexion. Si l'utilisateur saisit les bonnes informations, à savoir :

- identifiant : admin
- password : admin (et oui c'est très sécurisé, à ne pas reproduire)



The image shows a login form titled "Connexion" in a large, bold, brown font. Below the title is a white rectangular box with rounded corners containing the form fields. Inside the box, the label "Identifiant :" is followed by a text input field. Below that, the label "Mot de passe :" is followed by another text input field. At the bottom of the box is a green button with the text "Se connecter" in white.

Alors il sera redirigé vers la page secret qui affiche un code super secret:

Code secret : PHP ROCKS !

[Se déconnecter](#)

... et un lien pour pouvoir se déconnecter.

Attention :

Si on ne s'est pas connecté et qu'on essaie d'accéder malgré tout à la page "secret" alors on est redirigé vers la page d'accueil.

Quelques pistes :

- Si on se connecte, une session est créée
- ...donc si elle existe cette session et qu'elle contient la bonne valeur alors cela signifie que l'utilisateur est connecté !

A vous de jouer et de sécuriser ce code top secret !!!

Les Regex

Les Regex, qu'est ce que c'est ?

Les regex ou expressions régulières, sont des motifs de recherche qui permettent de valider et de manipuler des chaînes de caractère. Ici, nous allons voir leur utilité dans le cadre de la validation des mots de passe en PHP.

Syntaxe des regex

```
$regex = "/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[W_])\S{8,16}$/";
```

De prime abord, les regex impressionnent mais une fois qu'on a le bon dictionnaire, elles sont très faciles à décoder.

Une regex se place entre guillemets, commence et termine par deux barres obliques (slash) '/'.

- '^' : indique le début de la chaîne.
- '(?=.*[A-Z])' : recherche au moins une lettre majuscule.
- '(?=.*[a-z])' : recherche au moins une lettre minuscule.
- '(?=.*\d)' ou '(?=.*[0-9])' : recherche au moins un chiffre.
- '(?=.*[W_])' : recherche au moins un caractère spécial.
- '\S{8,16}' : correspond à une chaîne de caractères (S = string) de 8 à 16 caractères sans espaces.
- '\$' : indique la fin de la chaîne.

Utilisation dans PHP avec preg_match()

preg_match() et **preg_match_all()** sont des fonctions qui recherchent un ou des motifs spécifiés (en regex donc) dans une chaîne de caractères.

- **preg_match()** arrête ses recherches dès qu'il trouve une occurrence du motif spécifié.

Exemple d'utilisation de preg_match() dans la validation du mot de passe :

```
<?php
$password = "Exemple123!";
if (preg_match($regex, $password)):
    echo "Mot de passe valide !";
else:
```

```
    echo "Mot de passe invalide. Le mot de passe doit avoir une
longueur de 8 caractères ou plus et contenir au moins une majuscule,
une minuscule, un chiffre et un caractère spécial.";
endif;
?>
```

- **preg_match_all()** recherche toutes les occurrences du motif spécifié et les met dans un tableau. ('+' dans la regex indique que l'on recherche une ou plusieurs occurrences).

```
<?php
$regex = '/[0-9]+/';
$subject = 'J'ai 4 pommes et 6 bananes.';
preg_match_all($regex, $subject, $matches);

print_r($matches[0]);
```

Le résultat de \$matches[0] serait :

```
Array
(
    [0] => 4
    [1] => 6
)
```

Entraînez-vous à écrire vos propres regex sur ce site : <https://regex101.com/>

Petit rappel :

Global :

^ Début de la chaîne

\$ Fin de la chaîne

() Sert à faire un groupe

[] Sert à définir des énumérations. Exemple :

[a-z] = toutes les lettres minuscules de a à z

[a-zA-Z] = toutes les lettres minuscules et majuscules de a à z

{ } Servent à définir une longueur. Exemple :

[a-z]{3} = 3 lettres minuscules de a à z

[a-z]{3,10} = Entre 3 et 10 lettres minuscules de a à z

Seulement à l'intérieur d'une parenthèse :

. Sert à indiquer n'importe quel caractère

- | Sert à indiquer un "ou". Exemple :
(alb) sera valide pour la lettre a et la lettre b
- ? Sert à marquer qu'un groupe/element est optionnel.
- + Sert à indiquer qu'il y en a au moins 1
- * Sert à indiquer qu'il y en a 0 ou plusieurs

L'essentiel de la sécurité

1. La faille XSS

La faille XSS (pour cross-site scripting) est vieille comme le monde (euh, comme le Web) et on la trouve encore sur de nombreux sites web, même professionnels ! C'est une technique qui consiste à injecter du code HTML contenant du JavaScript dans vos pages pour le faire exécuter à vos visiteurs.

Exemple

Un formulaire sur votre site propose aux utilisateurs de déposer un commentaire sur l'article de blog que vous avez rédigé.

Un utilisateur mal-intentionné, va poster ceci dans le corps du commentaire :

```
<script>
while(true){
    alert("Votre PC s'auto-détruit dans 5 secondes");
}
</script>
```

Que va-t-il se passer ? Et bien le commentaire va être publié et le Javascript va s'exécuter. Les internautes auront donc une fenêtre popup qui s'ouvrira en

boucle leur indiquant un message pas très agréable et leur navigateur finira par planter pour boucle infinie.

Avec du Javascript, on peut faire tout et n'importe quoi, on peut supprimer la balise body, et hop, plus de site ! Pire, on peut faire une requête ajax qui irait interroger la partie backend et récupérer, pourquoi pas, l'ensemble des comptes utilisateurs du site. Ca devient très dangereux...

Comment la contrer ?

Et bien c'est très simple, la faille XSS se gère au moment d'afficher les informations, cela veut dire dans **les templates**. C'est à ce moment là que l'on va utiliser une fonction PHP "htmlspecialchars" afin de filtrer les caractères interdits et de les interpréter comme une simple chaîne de caractère.

Voici donc comment il est nécessaire d'afficher le commentaire de notre utilisateur mal intentionné :

```
<p>Commentaires:</p>
```

```
<p><?= htmlspecialchars($comment['description']) ?></p>
```

Ainsi, il vous suffit d'utiliser cette fonction **htmlspecialchars** sur toutes les données qui proviennent d'une **source** potentiellement dangereuse.

2. L'injection SQL

L'injection SQL est une faille de sécurité qui touche, comme son nom l'indique, au code SQL.

Il consiste à écrire du code SQL directement dans **les formulaires** ou dans les urls sensibles afin de le faire interpréter et d'accéder à des données présentes dans les bases de données.

Prenons l'exemple d'un formulaire de connexion:



The image shows a login form titled "Connexion". It contains two input fields: "Identifiant :" with the value "admin" and "Mot de passe :" with masked characters. Below the fields is a green button labeled "Se connecter".

L'identifiant pour se connecter est 'admin', donc logiquement pour aller chercher les informations en base de données concernant cet utilisateur afin de vérifier le mot de passe saisi, nous allons exécuter cette requête :

```
SELECT * FROM users WHERE username = $_POST['username']
```

// ce qui nous donnera :

```
SELECT * FROM users WHERE username = 'admin'
```

--> Cette requête nous retournera uniquement les informations de l'utilisateur "admin"

Maintenant, mettons nous à la place d'un pirate informatique et tentons quelque chose :



The image shows a web form titled "Connexion". It has two input fields: "Identifiant :" and "Mot de passe :". The "Identifiant" field contains the text "admin' OR '1'='1". The "Mot de passe" field is filled with 12 black dots. Below the fields is a green button labeled "Se connecter".

En SQL, voici la requête qui va alors s'exécuter si aucune précaution n'est prise :

```
SELECT * FROM users WHERE username = $_POST['username']
```

// ce qui nous donnera :

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1'
```

Cette requête ira chercher toutes les informations en table user pour les utilisateurs dont le username est "admin" OU pour ceux pour qui "1" est égal à "1", c'est à dire tous....

Comment contrer une injection SQL ?

Simplement en préparant les requêtes grâce à l'objet **PDO** vu précédemment, ainsi la requête potentiellement dangereuse deviendra :

// ce qui nous donnera :

```
SELECT * FROM users WHERE username = "admin' OR '1'='1"
```

Et il nous sera retourné les users dont le username est égal à **admin' OR '1'='1** soit aucun...

Pour rappel, voici le code pour préparer une requête :

```
$query = $pdo->prepare("SELECT * FROM users WHERE username = ?");  
$query->execute( [$_POST['username']] );  
$user = $query->fetchAll();
```

3. La sécurisation d'un espace utilisateur

Un espace utilisateur ne doit être accessible qu'à l'utilisateur connecté. Et celui-ci ne doit avoir accès qu'à ses propres données.

Pour faire cela, nous utiliserons **les sessions** vues précédemment.

Au début d'un fichier de traitement, par exemple "profil.php", avant toute autre action, il faudra vérifier que l'utilisateur est bien connecté :

```
session_start();
```

```
//si la session de l'utilisateur n'existe pas alors on redirige vers la page de connexion  
if(!isset($_SESSION['user'])) {  
    header('location: login.php');  
    exit;  
}
```

Cela signifie que lorsque l'utilisateur va tenter de se connecter, il nous faudra, dans un premier temps, vérifier qu'il a saisi les bonnes informations (identifiant et mot de passe) et ensuite, si c'est le cas, nous créerons cette session qui permettra de garder en mémoire le fait que l'utilisateur s'est connecté.

C'est cette même session qui contiendra les informations de l'utilisateur et par exemple son id qui nous servira plus tard à aller chercher UNIQUEMENT les informations qui le concernent.

Un exemple :

```
$_SESSION['user']['id'] = $user['id'];  
$_SESSION['user']['nom'] = $user['nom'];  
$_SESSION['user']['prenom'] = $user['prenom'];  
$_SESSION['user']['email'] = $user['email'];
```

Ici, on imagine que \$user est la variable qui contient les informations provenant de la base de données. On crée alors la session 'user' dans laquelle nous mettons toutes les informations qui nous intéressent.

4. Le hachage des mots de passe

Pourquoi crypter les mots de passe ?

Le hachage des mots de passe, et plus généralement des données sensibles de l'utilisateur, est une étape **essentielle et basique** de sécurité. Sans hachage, il est très facile pour un attaquant de récupérer le mot de passe et d'accéder frauduleusement à votre application, ou à d'autres, si l'utilisateur utilise ce mot de passe plusieurs fois.

Quelle méthode pour hacher ?

Il existe beaucoup de méthodes de hachage, mais toutes n'ont pas le même but et ne sont pas adapter aux mêmes opérations.

En PHP, il y a plusieurs fonctions qui génèrent un algorithme permettant de hachage des données mais beaucoup sont devenues obsolètes car trop rapides à calculer ou présentant des risques de collisions. Ainsi, si vous croisez **les fonctions md5()** ou **sha1()** dans du code, vous êtes face à des données mal protégées voire non protégées. Ici, nous allons voir la méthode recommandée actuellement et qui se base sur l'algorithme **bcrypt**.

Le hachage du mot de passe avec bcrypt

bcrypt est un algorithme complexe et heureusement pour nous il est intégré nativement au sein d'une API dans PHP et donc accessible via des fonctions précises.

Il permet de "hash" les mots de passe, c'est à dire de chiffrer de manière irréversible une chaîne de caractères.

Deux fonctions sont nécessaires pour hash un mot de passe :

- **password_hash()** qui prend en entrée la variable contenant le mot de passe en clair. `$password = "monMotDePasseUltraSecurise";`
- `$hash = password_hash($password, PASSWORD_DEFAULT);`
-
- **password_verify()** qui est utilisée pour vérifier si un mot de passe correspond à son hash sécurisé. Elle prend en entrée le mot de passe en clair et le hash à vérifier. `if (password_verify($password, $hash)):`
- `?>`
- `Le mot de passe est correct !`
- `<?php else: ?>`
- `Le mot de passe est incorrect.`
- `<?php endif; ?>`
-

En utilisant conjointement ces deux fonctions, on sécurise les mots de passe dans la base de données. Attention, ils sont toujours vulnérables à d'autres attaques, d'autant plus si ils sont "faibles". D'où l'importance de sensibiliser vos utilisateurs à la politique des mots de passe.

Pour aller plus loin, le site de l'**ANSSI** permet de calculer la force d'un mot de passe et donne les recommandations officielles :

<https://www.ssi.gouv.fr/administration/precautions-elementaires/calculer-la-force-dun-mot-de-passe/>

5. La faille CSRF

Qu'est ce que c'est ?

La faille CSRF (Cross-Site Request Forgery, en français, falsification de requêtes inter-sites) est une vulnérabilité de sécurité qui permet à un attaquant de forcer un utilisateur connecté à effectuer des actions non voulues sur un site web sans que l'utilisateur en ait conscience. Cela se produit lorsque l'application ne vérifie pas correctement l'origine de la requête.

Les conséquences peuvent être graves : suppression de données, transactions non voulues, modification de paramètres.

Il est indispensable de se prémunir contre cette faille.

Comment ça marche la faille CSRF ?

Un utilisateur est connecté à un site web (par exemple, une banque en ligne).

Il visite un site malveillant.

Ce site malveillant déclenche des requêtes vers le site de la banque au nom de l'utilisateur connecté, exploitant sa session.

Comment protéger nos applications contre cette faille ?

Les tokens anti-CSRF

C'est la méthode *recommandée et essentielle* à mettre en place.

Un token est une valeur unique, générée par le serveur, qui est associée à une session utilisateur et incluse dans **les formulaires** web ou les requêtes AJAX. Ce token permet de vérifier que la requête provient d'une **source** légitime, c'est-à-dire du même site web, et qu'elle n'est pas le résultat d'une manipulation externe.

1. Génération du Token : Lorsqu'un utilisateur se connecte à un site web, un token anti-CSRF est généré côté serveur et associé à sa session. Ce token est stocké côté serveur et dans un cookie ou une balise cachée côté client.

2. Inclusion dans **les formulaires** : Lorsque l'utilisateur soumet un formulaire sur le site web, le token anti-CSRF est inclus en tant que champ caché dans le formulaire ou dans l'en-tête de la requête AJAX.
3. Vérification côté serveur : Lorsque le serveur reçoit la requête, il vérifie si le token anti-CSRF inclus correspond à celui associé à la session de l'utilisateur. Si les tokens correspondent, la requête est considérée comme légitime et traitée.
4. Rejet des requêtes non autorisées : Si les tokens ne correspondent pas, le serveur rejette la requête, empêchant ainsi les attaques CSRF.

Comment créer et utiliser les tokens anti-CSRF?

random_bytes() est une fonction de PHP qui génère une chaîne de caractères aléatoire cryptographiquement sécurisée.

bin2hex() transforme cette chaîne en représentation hexadécimale plus lisible, ce qui facilite le stockage, l'affichage et la manipulation de ces données.

Donc pour générer un token :

```
<?php
```

```
function generateCSRFToken() : string {  
    // Générer une chaîne de caractères aléatoire de 32 octets (256 bits)  
  
    $token = bin2hex(random_bytes(32));  
  
    return $token;  
}  
  
// Exemple d'utilisation pour générer un token  
  
$csrfToken = generateCSRFToken();  
  
// Stocker ce token dans la session de l'utilisateur ou dans un cookie  
  
$_SESSION['csrf_token'] = $csrfToken;
```

hash_equals() est la fonction qui permet de comparer le token envoyé dans la requête et celui stocké par le serveur. On préférera utiliser **hash_equals()** à **==** pour éviter les attaques de type "timing attack".

```
<?php
```

```
function validateCSRFToken($token) : bool {  
    if (isset($_SESSION['csrf_token']) &&  
hash_equals($_SESSION['csrf_token'], $token)) {  
  
        // Le token est valide, procéder au traitement de la  
requête  
  
        return true;  
    } else {  
  
        // Le token est invalide, rejeter la requête  
  
        return false;  
    }  
}  
  
// Exemple d'utilisation pour vérifier un token  
  
if (isset($_POST['csrf_token']) &&  
validateCSRFToken($_POST['csrf_token'])) {  
  
    // Le token est valide, traiter la requête  
  
} else {  
  
    // Le token est invalide, prendre des mesures pour l'attaque  
CSRF  
  
}
```

L'inclusion d'une vérification de token anti-CSRF est essentielle chaque fois qu'une action sensible ou modifiant l'état de l'application est effectuée. Voici quelques endroits clés où vous devez inclure la vérification de token anti-CSRF dans votre application web :

- Changement de mot de passe : Lorsque l'utilisateur souhaite changer son mot de passe, vous devriez inclure une vérification de token anti-CSRF.
- Soumission de formulaires sensibles : Tout formulaire qui permet à l'utilisateur de modifier des données sensibles, telles que les informations de compte, les données financières, ou les paramètres de confidentialité, doit inclure une vérification de token anti-CSRF.
- Transferts d'argent : Si votre application gère des transactions financières, assurez-vous d'inclure la vérification de token anti-CSRF lors de la soumission de formulaires liés aux transferts d'argent ou aux paiements.
- Actions de suppression : Les actions permettant de supprimer des données, comme la suppression de comptes utilisateur ou la suppression de messages, doivent également être protégées.
- Toute action nécessitant une authentification : En général, toutes les actions qui nécessitent que l'utilisateur soit authentifié devraient être protégées contre les attaques CSRF, car un attaquant pourrait exploiter la session en cours de l'utilisateur pour mener des actions malveillantes.
- Requêtes AJAX sensibles : Si votre application utilise des requêtes AJAX pour effectuer des actions sensibles, assurez-vous d'inclure la vérification de token anti-CSRF dans ces requêtes.
- Modification de paramètres de compte : Les actions liées à la modification des paramètres de compte, tels que l'adresse e-mail, le numéro de téléphone ou les préférences de compte, doivent également être protégées.

En règle générale, tout ce qui a le potentiel de modifier l'état de l'application ou de causer des dommages importants en cas d'exploitation doit être protégé par la vérification de token anti-CSRF.

N'oubliez pas que la vérification de token anti-CSRF doit être effectuée côté serveur, car c'est le serveur qui détient la **source** de vérité pour le token associé à la session de l'utilisateur. Les tokens doivent être générés lors de l'authentification de l'utilisateur et vérifiés à chaque action sensible.